

Medical Appointment System API Documentation

Overview

This comprehensive REST API for a medical appointment booking system is built using **Express.js**, a fast, unopinionated, minimalist web framework for Node.js. Express.js provides a robust set of features for web and mobile applications, making it ideal for building RESTful APIs that serve as the backbone of modern healthcare management systems.

Key Features of Express.js:

Routing: Express.js provides a powerful routing mechanism that allows developers to handle different HTTP methods (GET, POST, PUT, DELETE) and URL patterns efficiently. This enables clean, RESTful API design with intuitive endpoint structures.

Middleware: Express supports middleware functions that can execute code, modify request and response objects, end the request-response cycle, and call the next middleware function in the stack. This is crucial for implementing authentication, logging, error handling, and request processing.

Template Engines: While primarily used for APIs, Express supports various template engines for dynamic content rendering when needed for administrative interfaces or documentation.

Static Files: Built-in support for serving static files makes it easy to serve documentation, images, or other assets directly from the server.

Error Handling: Comprehensive error handling mechanisms ensure robust API behavior and meaningful error responses to clients.

This API serves as a healthcare management system backend, providing endpoints for managing patients, doctors, appointments, medical tests, and notifications. The system supports three user types: Patients, Doctors, and Assistants, each with specific roles and permissions.

Base URL: `http://localhost:3000`

Installed Libraries

Core Dependencies

Library	Version	Role & Purpose
express	^5.1.0	Main web framework for building the REST API server. Handles routing, middleware, and HTTP request/response processing.
cors	^2.8.5	Enables Cross-Origin Resource Sharing (CORS) for frontend-backend communication. Essential for allowing web applications to make requests to the API from different domains.
mysql2	^3.14.1	Modern MySQL database driver for Node.js. Provides connection pooling, prepared statements, and async/await support for database operations.
dotenv	^16.5.0	Loads environment variables from .env file for secure configuration management. Keeps sensitive data like database credentials and JWT secrets out of source code.

Authentication & Security

Library	Version	Role & Purpose
bcrypt	^6.0.0	Primary password hashing library using the bcrypt algorithm. Provides secure password storage with salt generation and verification. Industry standard for password security.
bcryptjs	^3.0.2	Pure JavaScript implementation of bcrypt. Serves as a fallback or alternative to the native bcrypt library, ensuring compatibility across different environments.
jsonwebtoken	^9.0.2	Creates and verifies JSON Web Tokens (JWT) for stateless user authentication. Enables secure user sessions without server-side session storage.

Utilities

Library	Version	Role & Purpose
uuid	^11.1.0	Generates unique identifiers (UUIDs) for appointments, references, and other entities requiring unique identification. Ensures collision-free ID generation.
multer	^2.0.1	Handles multipart/form-data for file uploads. Essential for processing test result uploads and other file attachments in the medical system.

Development

Library	Version	Role & Purpose
nodemon	^3.1.10	Development tool that automatically restarts the Node.js server when file changes are detected. Improves development efficiency by eliminating manual server restarts.

How to Run the Server

Prerequisites

- **Node.js** (v14 or higher)
- **MySQL** database server

- **npm** or **yarn** package manager

Installation Steps

1. Clone the repository (if applicable)

```
bash  
  
git clone [repository-url]  
cd backend
```

2. Install dependencies

```
bash  
  
npm install
```

3. Environment Configuration

Create a `.env` file in the root directory with the following variables:

```
env  
  
PORT=3000  
JWT_SECRET=your_jwt_secret_key_here  
DB_HOST=localhost  
DB_USER=your_database_username  
DB_PASSWORD=your_database_password  
DB_NAME=your_database_name
```

4. Database Setup

- Ensure MySQL server is running
- Create the required database and tables
- Configure database connection in `config/db.js`

5. Start the Server Development Mode:

```
bash  
  
npm start
```

This uses nodemon for automatic server restart on file changes. **Production Mode:**

```
bash  
  
node index.js
```

6. Verify Server is Running

```
Server running on http://localhost:3000
```

Authentication Endpoints (Detailed)

1. Register User

Method: POST

Full URL: `http://localhost:3000/auth/register`

Access: Public

Description: This endpoint allows new users to create an account in the system. It supports registration for three different user types: Patient, Doctor, and Assistant.

How Registration Works:

Step 1: Input Validation The system validates that all required fields are provided:

- `fullName`: User's full name
- `phoneNumber`: Unique phone number (used as username)
- `password`: User's password
- `userType`: Role type ('Patient', 'Doctor', 'Assistant')

Step 2: Duplicate Check Queries the database to ensure the phone number isn't already registered:

sql

```
SELECT user_id FROM Users WHERE phone_number = ?
```

Step 3: Password Hashing Uses bcrypt to securely hash the password:

javascript

```
const salt = await bcrypt.genSalt(10);
const passwordHash = await bcrypt.hash(password, salt);
```

- Generates a salt with complexity level 10
- Hashes the password with the salt for security

Step 4: User Creation Inserts the new user into the Users table:

sql

```
INSERT INTO Users (full_name, phone_number, password_hash, user_type) VALUES (?, ?, ?,
```

Step 5: Role-Specific Table Based on userType, creates an entry in the corresponding table:

- Patient → `INSERT INTO Patients (patient_id) VALUES (?)`
- Doctor → `INSERT INTO Doctors (doctor_id) VALUES (?)`
- Assistant → `INSERT INTO Assistants (assistant_id) VALUES (?)`

Step 6: JWT Token Generation Creates a JWT token for immediate authentication:

javascript

```
const token = jwt.sign(  
  { id: userId, userType },  
  JWT_SECRET,  
  { expiresIn: '7d' }  
);
```

Request Body:

json

```
{  
  "fullName": "John Doe",  
  "phoneNumber": "1234567890",  
  "password": "securePassword123",  
  "userType": "Patient"  
}
```

Response (201 Created):

json

```
{  
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  
  "user": {  
    "id": 1,  
    "fullName": "John Doe",  
    "userType": "Patient"  
  }  
}
```

2. Login User

Method: POST

Full URL: `http://localhost:3000/auth/login`

Access: Public

Description: This endpoint authenticates existing users by validating their phone number and password combination.

How Login Works:

Step 1: Input Validation Ensures phone number and password are provided.

Step 2: User Lookup Queries the database to find the user:

sql

```
SELECT user_id, full_name, password_hash, user_type FROM Users WHERE phone_number = ?
```

Step 3: Password Verification Uses bcrypt to compare the provided password with the stored hash:

javascript

```
const isMatch = await bcrypt.compare(password, user.password_hash);
```

- bcrypt handles the salt extraction and comparison automatically
- Returns true if passwords match, false otherwise

Step 4: JWT Token Generation If authentication succeeds, generates a new JWT token:

javascript

```
const token = jwt.sign(  
  { id: user.user_id, userType: user.user_type },  
  JWT_SECRET,  
  { expiresIn: '7d' }  
);
```

JWT Token Structure:

- **Header:** Contains token type (JWT) and signing algorithm (HS256)
- **Payload:** Contains user ID, user type, and expiration time
- **Signature:** Ensures token integrity using the JWT_SECRET

Request Body:

json

```
{  
  "phoneNumber": "1234567890",  
  "password": "securePassword123"  
}
```

Response (200 OK):

json

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
    "fullName": "John Doe",
    "userType": "Patient"
  }
}
```

3. Forgot Password

Method: POST

Full URL: `http://localhost:3000/auth/forgot-password`

Access: Public

Description: This endpoint handles password reset requests. It checks if a user exists with the provided phone number and initiates the password reset process.

How Forgot Password Works:

Step 1: Phone Number Validation Ensures phone number is provided.

Step 2: User Verification Checks if user exists in database.

Step 3: Security Response Always returns success message (prevents user enumeration attacks).

Step 4: SMS Integration Would trigger SMS with reset code (implementation placeholder).

Request Body:

json

```
{
  "phoneNumber": "1234567890"
}
```

Response (200 OK):

json

```
{
  "msg": "If an account with that phone number exists, a password reset code has been sent."
}
```

Appointment Management Endpoints

4. Create Appointment

Method: POST

Full URL: (`http://localhost:3000/appointment`)

Access: Private (Patient only)

Description: This endpoint allows patients to book new medical appointments. It calculates the total cost based on selected tests, creates the appointment record, and links the selected medical tests to the appointment.

How it works:

1. Validates all required fields including patientId, addressId, appointmentDatetime, and testIds array
2. Queries the Medical_Tests table to get prices for all selected tests
3. Calculates the total cost by summing up individual test prices
4. Generates a unique appointment reference ID using UUID v4
5. Inserts the appointment record into the Appointments table with all details and calculated total cost
6. For each test ID in the testIds array, creates a record in the Appointment_Tests junction table
7. Returns the internal appointment ID and public reference ID for tracking

5. Get Appointment Details

Method: GET

Full URL: (`http://localhost:3000/appointment/:id`)

Access: Private

Description: This endpoint retrieves comprehensive details about a specific appointment including patient information, address details, scheduled tests, and appointment status.

How it works:

1. Takes the appointment ID from the URL parameters
2. Performs a complex JOIN query between Appointments and Addresses tables to get full address details
3. Concatenates address components into a readable format
4. Executes a second query to get all tests associated with the appointment
5. Returns comprehensive appointment information or 404 if appointment not found

6. Cancel Appointment

Method: PUT

Full URL: (`http://localhost:3000/appointment/:id/cancel`)

Access: Private

Description: This endpoint allows users to cancel an existing appointment by updating its status to 'Cancelled'.

How it works:

1. Extracts the appointment ID from the URL parameters
2. Executes an UPDATE query to change the appointment status to 'Cancelled'
3. Does not delete the appointment record, preserving it for audit purposes
4. Returns a confirmation message indicating successful cancellation

7. Upload Test Results

Method: POST

Full URL: (`http://localhost:3000/appointment/:id/results`)

Access: Private (Admin/Doctor)

Description: This endpoint allows authorized personnel to upload test result files for completed appointments.

How it works:

1. Validates that both testId and resultFileUrl are provided
 2. Inserts a new record into the Test_Results table linking the appointment, specific test, and file URL
 3. Creates an audit trail of when results were uploaded
 4. Returns confirmation of successful upload
-

Patient Management Endpoints

8. Get Patient Profile

Method: GET

Full URL: (`http://localhost:3000/patient/:id`)

Access: Private

Description: This endpoint retrieves complete profile information for a specific patient, including personal details and medical information.

How it works:

1. Takes the patient ID from URL parameters
2. Performs an INNER JOIN between Users and Patients tables to get complete profile data
3. Retrieves general user information and patient-specific data
4. Returns 404 if no patient found with the given ID

9. Update Patient Profile

Method: PUT

Full URL: (`http://localhost:3000/patient/:id`)

Access: Private

Description: This endpoint allows patients to update their profile information including name, date of birth, and gender.

How it works:

1. Extracts patient ID from URL parameters and update data from request body
2. Executes UPDATE query on Users table to update the full_name field
3. Executes UPDATE query on Patients table to update date_of_birth and gender
4. Returns confirmation message upon successful completion

10. Get Patient Addresses

Method: GET

Full URL: (`http://localhost:3000/patient/:id/addresses`)

Access: Private

Description: This endpoint retrieves all saved addresses for a specific patient.

How it works:

1. Takes the patient ID from URL parameters
2. Queries the Addresses table filtering by patient_id
3. Returns all address records associated with the patient
4. Includes the is_default flag to identify the primary address

11. Add Patient Address

Method: POST

Full URL: (`http://localhost:3000/patient/:id/addresses`)

Access: Private

Description: This endpoint allows patients to add new addresses to their account.

How it works:

1. Extracts patient ID from URL and address details from request body
2. Sets is_default to false if not provided in the request
3. Inserts new address record into Addresses table with patient_id link
4. Returns confirmation message upon successful insertion

Doctor Management Endpoints

12. Get All Verified Doctors

Method: GET

Full URL: (`http://localhost:3000/doctor`)

Access: Public

Description: This endpoint returns a list of all doctors who have been verified by the system administrators.

How it works:

1. Performs INNER JOIN between Doctors and Users tables
2. Applies WHERE clause to filter only doctors with is_verified = TRUE
3. Returns doctor ID, full name, and featured status
4. Public access allows patients to browse available doctors

13. Get Doctor Profile

Method: GET

Full URL: (`http://localhost:3000/doctor/:id`)

Access: Public

Description: This endpoint retrieves detailed profile information for a specific doctor including their clinic address and verification status.

How it works:

1. Takes doctor ID from URL parameters
2. Joins Doctors and Users tables to get comprehensive doctor information
3. Returns detailed profile including name, clinic location, and verification status
4. Public access allows patients to research doctors before booking

14. Update Doctor Profile

Method: PUT

Full URL: (`http://localhost:3000/doctor/:id`)

Access: Private (Doctor only)

Description: This endpoint allows doctors to update their profile information, specifically their clinic address.

How it works:

1. Extracts doctor ID from URL parameters and new clinic address from request body
2. Updates the clinic_address field in the Doctors table
3. Authentication middleware ensures only the doctor themselves can update their profile

4. Returns confirmation message upon successful update

15. Get Doctor Appointments

Method: GET

Full URL: (`http://localhost:3000/doctor/:id/appointments`)

Access: Private (Doctor only)

Description: This endpoint allows doctors to view all appointments scheduled with them.

How it works:

1. Takes doctor ID from URL parameters
 2. Joins Appointments table with Users table to get patient names
 3. Filters appointments where doctor_id matches the provided ID
 4. Orders results by appointment_datetime in descending order
 5. Returns list of appointments with basic information
-

Assistant Management Endpoints

16. Get Assistant Profile

Method: GET

Full URL: (`http://localhost:3000/assistant/:id`)

Access: Private (Assistant only)

Description: This endpoint allows assistants to view their own profile information.

How it works:

1. Validates that the requesting user is an Assistant and matches the requested ID
2. Returns 403 Forbidden if user type is not Assistant or ID doesn't match
3. Joins Users and Assistants tables to get complete profile information
4. Returns basic profile information including ID, name, and contact details

17. Get Assistant Appointments

Method: GET

Full URL: (`http://localhost:3000/assistant/:id/appointments`)

Access: Private (Assistant only)

Description: This endpoint allows assistants to view all appointments assigned to them.

How it works:

1. Validates that requesting user is an Assistant and matches the requested ID
2. Returns 403 Forbidden if validation fails

3. Joins Appointments table with Users table to get patient names
 4. Filters appointments by assistant_id to show only assigned appointments
 5. Orders results by appointment_datetime in descending order
-

Medical Tests Endpoints

18. Get All Tests

Method: GET

Full URL: (`http://localhost:3000/test`)

Access: Public

Description: This endpoint returns a list of all available medical tests that patients can book.

How it works:

1. Queries the Medical_Tests table for all available tests
2. Applies WHERE clause to filter only active tests (is_active = 1)
3. Returns essential information: test ID, code, name, and price
4. Public access allows patients to browse available tests before booking

19. Get Test Details

Method: GET

Full URL: (`http://localhost:3000/test/:id`)

Access: Public

Description: This endpoint provides comprehensive information about a specific medical test including detailed description, preparation instructions, and expected turnaround time.

How it works:

1. Takes test ID from URL parameters
 2. Queries Medical_Tests table for the specific test with comprehensive details
 3. Returns all available information about the test including preparation requirements
 4. Returns 404 if test not found with given ID
 5. Helps patients understand what to expect and how to prepare
-

Notification Endpoints

20. Get User Notifications

Method: GET

Full URL: (`http://localhost:3000/notification/user/:userId`)

Access: Private

Description: This endpoint retrieves all notifications for a specific user, ordered by most recent first.

How it works:

1. Takes user ID from URL parameters
2. Queries Notifications table filtering by user_id
3. Orders results by created_at in descending order
4. Returns notification ID, title, message, and creation timestamp
5. Authentication ensures users can only access their own notifications

21. Mark Notification as Read

Method: PUT

Full URL: (`http://localhost:3000/notification/:id/read`)

Access: Private

Description: This endpoint allows users to mark specific notifications as read, helping them track which notifications they have already seen.

How it works:

1. Extracts notification ID from URL parameters
2. Updates the is_read field to 1 (true) in the Notifications table
3. Uses the notification ID to target the specific notification
4. Returns confirmation message upon successful update
5. Client applications can use this to show read/unread states

Error Handling

The API uses standard HTTP status codes for consistent error handling:

- **200 OK:** Request successful
- **201 Created:** Resource created successfully
- **400 Bad Request:** Missing or invalid parameters
- **401 Unauthorized:** Invalid or missing authentication
- **403 Forbidden:** User lacks permission for this resource
- **404 Not Found:** Requested resource doesn't exist
- **500 Internal Server Error:** Server-side error occurred

Error Response Format:

```
json
```

```
{  
  "msg": "Descriptive error message explaining what went wrong"  
}
```

Authentication & Security

JWT Token Authentication

Most endpoints require JWT authentication. Include the token in requests:

```
http
```

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Token Contents:

- **User ID** (for identifying the user)
- **User Type** (Patient/Doctor/Assistant for role-based access)
- **Expiration** (7 days from issuance)

Security Features Implemented:

1. **Password Security:** Bcrypt hashing with salt (strength: 10)
2. **SQL Injection Protection:** Parameterized queries throughout
3. **Role-Based Access Control:** Different permissions per user type
4. **Input Validation:** Required field validation on all endpoints
5. **Phone Number Enumeration Protection:** Generic responses for security-sensitive operations

Database Schema Overview

Core Tables:

- **Users:** Central user information (ID, name, phone, password hash, user type)
- **Patients:** Patient-specific data (date of birth, gender)
- **Doctors:** Doctor-specific data (clinic address, verification status, featured status)
- **Assistants:** Assistant-specific data
- **Appointments:** Appointment bookings with patient, address, datetime, costs
- **Medical_Tests:** Available tests with codes, names, prices, descriptions
- **Appointment_Tests:** Junction table linking appointments to multiple tests

- **Addresses:** Patient addresses with default address capability
 - **Notifications:** User notifications with read status tracking
 - **Test_Results:** Links test result files to specific appointments and tests
-

Production Deployment Recommendations

For production environments, consider implementing:

1. **Rate Limiting:** Prevent API abuse with request throttling
2. **Request Logging:** Comprehensive logging for monitoring and debugging
3. **Input Sanitization:** Additional validation beyond current basic checks
4. **HTTPS Enforcement:** Secure data transmission
5. **Environment Variables:** Proper configuration management
6. **Database Connection Pooling:** Optimize database performance
7. **Error Monitoring:** Real-time error tracking and alerting
8. **API Documentation:** Interactive documentation (Swagger/OpenAPI)
9. **Backup Strategy:** Regular database backups and recovery procedures
10. **Load Balancing:** Handle increased traffic with multiple server instances